

Proyecto NanoFiles



Gabriel Benavides Muñoz

DNI: 49170069W

Subgrupo: 3.3

David Molina-Niñirola Jiménez

DNI: 49444891C

Subgrupo: 3.3

Profesor:

Juan José Pujante Moreno

1. Introducción.	3
2. Formato de los mensajes del protocolo de comunicación con el Directorio	3
3. Formato de los mensajes del protocolo de transferencia de ficheros	8
4. Autómatas de protocolo	11
5. Ejemplo de intercambio de mensajes	14
6. Mejoras Implementadas	19
7. Captura de Pantalla de WireShark	20
8. Conclusión	21

1. Introducción.

En este documento se especifica el diseño de los protocolos de comunicación UDP entre el Directorio y Usuarios que actuarán como Clientes y Servidores, y la comunicación TCP entre Peers (Cliente y Servidor)

Las mejoras que hemos decidido implementar son “**diridl avanzado**” y “**quit avanzado**”.

2. Formato de los mensajes del protocolo de comunicación con el Directorio

Para definir el protocolo de comunicación con el *Directorio*, vamos a utilizar mensajes textuales con formato “campo:valor”. El valor que tome el campo “operation” (código de operación) indicará el tipo de mensaje y por tanto su formato (qué campos vienen a continuación).

Tipos y descripción de los mensajes:

Familia ‘ping’

Mensaje: **ping**

Sentido de la comunicación: Cliente → Directorio

Descripción: El cliente envía un mensaje vacío y espera confirmación para comprobar que el Directorio sigue activo y es capaz de conversar.

Ejemplo:

```
operation: ping\nprotocol: 69696969F\n\n
```

Mensaje: **pingOk**

Sentido de la comunicación: Directorio → Cliente

Descripción: El directorio le va a confirmar al cliente que se ha establecido la conexión de forma exitosa y el protocolo que usan ambos es el mismo.

Ejemplo:

```
operation: pingok\n\n
```

Mensaje: **pingDenied**

Sentido de la comunicación: Directorio → Cliente

Descripción: El directorio le va a confirmar al cliente que no se ha podido establecer la conexión, debido a un protocolo erróneo.

Ejemplo:

```
operation: pingdenied\n\n
```

Familia 'serve' y 'quit'

Mensaje: **serve**

Sentido de la comunicación: Servidor → Directorio

Descripción: Un servidor le comunica al Directorio que va a empezar a estar a la escucha de una conexión TCP para servir a otros clientes (La IP del Servidor se conoce ya por la propia conexión establecida, pero el Servidor debe proporcionar un Puerto de escucha.)

Ejemplo:

```
operation: serve\nnickname : pepito_el_grillo\nport: 10000\n\n
```

Mensaje: **serveOk**

Sentido de la comunicación: Servidor→ Directorio

Descripción: El Directorio reconoce que un usuario va a actuar como Servidor, enviando este mensaje una vez guarde sus datos (nickname, Ip y puerto) en su buffer para comunicarle que ha sido aceptado.

Ejemplo:

```
operation: serveOk\nIP: 183.3.21.24\nport: 10000\n\n
```

Mensaje: **quit**

Sentido de la comunicación: Servidor → Directorio

Descripción: El Servidor decide parar de estar a la escucha de otros peers logueados, y le pide al Directorio que le elimine de su lista de peers conectados.

Ejemplo:

```
operation: quit\n\n
```

Mensaje: **quitOk**

Sentido de la comunicación: Directorio → Servidor

Descripción: El Directorio reconoce la petición del Servidor a parar de ser un Servidor de ficheros. Le habrá eliminado de su lista de Servidores registrados antes de enviar este mensaje de confirmación.

Ejemplo:

```
operation : quitOk\n\n
```

Familia 'peers'

Mensaje: **peers**

Sentido de la comunicación: Cliente → Directorio

Descripción: El Cliente le pregunta al Directorio cuáles son los peers que están actuando como Servidor a la escucha de Clientes, que estén logueados y hayan otorgado un puerto de escucha.

Ejemplo:

```
operation: peers\n\n
```

Mensaje: **peersList**

Sentido de la comunicación: Directorio → Cliente

Descripción: El Directorio va a enviar una lista de todos los peers logueados (que han dado un nickname único y han establecido conexión con el mismo protocolo) que hayan hecho un 'serve' previamente, ofreciéndose como Servidor a la escucha de Clientes.

Ejemplo:

```
operation: peersList\nnickname: pato_donald\nIP: 127.90.12.4\nport: 10000\nnickname: pepito_el_grillo\nIP: 192.60.11.209\nport: 5003\n\n
```

Familia 'dirfiles'

Mensaje: **dirfiles**

Sentido de la comunicación: Cliente → Directorio

Descripción: El Cliente pide al Directorio conocer la lista de los archivos que tiene guardado en su buffer.

Ejemplo:

```
operation: dirfiles\n\n
```

Mensaje: **dirfilesList**

Sentido de la comunicación: Directorio → Cliente

Descripción: El Directorio enviará al Cliente una lista con todos los archivos que tenga guardados.

Ejemplo:

```
operation: dirfilesList\nnumber_files: 2\nfile: hola.txt\nsize: 12242\nhash_substring: a1b2c3\nfile: adios.pdf\nsize: 1953135\nhash_substring: b2b3b4\n\n
```

Familia 'dirdl':

Mensaje: **dirdl <subcadena_hash>**

Sentido de la comunicación: Cliente → Directorio

Descripción: El Cliente le pide al Directorio que le deje descargar el fichero que él posee, cuya subcadena hash es la introducida como parámetro.

Ejemplo:

```
operation: dirdl\n
hash_substring: 1b3cd\n
\n
```

Mensaje: **dirdlOk**

Sentido de la comunicación: Directorio → Cliente

Descripción: El Directorio encuentra el archivo que el Cliente ha pedido en su propia lista de ficheros, por lo que le envía un mensaje de confirmación y envía los datos del fichero pedido.

Ejemplo:

```
operation: dirdlOk\n
file: mininet.ova\n
data: <bytes_de_data_que_componen_el_fichero>\n
\n
```

Mensaje: **dirdlClientOk**

Sentido de la comunicación: Cliente → Directorio

Descripción: Tras recibir que en directorio tiene el archivo que previamente le hemos pedido, el cliente tendrá que mandar este mensaje para que este le empiece a mandar el contenido del fichero.

Ejemplo:

```
operation: dirdlClientOk\n
\n
```

Mensaje: **dirdlFailed**

Sentido de la comunicación: Directorio → Cliente

Descripción: El Directorio le avisa al Cliente que no puede proveer el archivo de la subcadena hash que ha enviado.

Sugerimos implementar un campo:valor de tipo 'error:byte' para no hacer dos mensajes: uno para 'no se encuentra este fichero' (0x01) y otro para 'hay dos ficheros con esta subcadena hash' (0x02).

Ejemplo:

```
operation: dirdlFailed\n
error: 0x01\n
\n
```

3. Formato de los mensajes del protocolo de transferencia de ficheros

Para definir el protocolo de comunicación con un servidor de ficheros, vamos a utilizar mensajes binarios multiformato. El valor que tome el campo "opcode" (código de operación) indicará el tipo de mensaje y por tanto cuál es su formato, es decir, qué campos vienen a continuación.

Exceptuando los Opcodes de control orientados a mensajes de error, todos los demás poseerán Opcodes que representan el orden que seguirán en una 'conversación normal', y aquellos que sean de Control para confirmar serán equivalentes a sumar el Opcode del mensaje causante, sumando 16. (0x01 primero (mensaje), luego 0x11 (mensajeOk), y 0x21 (mensajeSiguiente)..., por ejemplo.)

PD: Hemos visto este fragmento de uno de los boletines:

"A continuación se proporciona un conjunto de ejemplos, los cuales no tienen por qué usarse en todas las circunstancias y pueden ser modificados, para ilustrar el contexto de una comunicación con el servidor de ficheros."

Este es un fragmento que posteriormente enseñaba los tipos de mensajes binarios (Control, Operación, TLV). Hemos decidido que las respuestas que pueda dar el servidor necesitan más campos, así que hemos sugerido modificarlas.

Tipos y descripción de los mensajes

Mensaje: **peerfiles <peer_nickname> (pide lista ficheros) (opcode = 0x01)**
(Control)

Sentido de la comunicación: Cliente → Servidor de ficheros

Descripción: El Cliente pide al Servidor con el nickname seleccionado que envíe una lista de los archivos que tiene disponibles para descargar.

Ejemplo:

Opcode (1 byte)
0x01

Mensaje: **filelist (manda la lista de ficheros) (opcode =0x11)**

Sentido de la comunicación: Servidor de ficheros → Cliente

Descripción: El Servidor de ficheros proporciona una lista de los ficheros que posee para ser descargados por el cliente.

El tipo de mensaje que pensamos usar aquí es distinto de los tres modelos planteados en el boletín 6. Pensamos que es mejor que todo se envíe de golpe, incluso si el mensaje de por sí requiere fragmentación.

El modelo diseñado es parecido a juntar 2 TLV en un mensaje por cada fichero que el servidor envía, NumFiles indicando cuántas veces se repetirá esta última parte, y además como guía al Cliente para saber cuántas veces va a extraer esos campos.

Ejemplo:

Opcode (1 byte)	Numfiles (4 bytes)	nameLength (4 bytes)	fileName (N bytes)	hashLength (4 bytes)	hashSubstring (M byte)	fileSize (8 bytes)
0x11	0x01	0x08	hola.pdf	0x00 0x00 0xF1 0xF6	0x03	3446

Los campos que no varían son el Opcode y Numfiles, los demás varían dependiendo del número de fichero que estemos viendo y por cada uno tendrá los demás campos. Numfiles es necesario ya que PeerMessage tiene que saber la cantidad de mensajes para inicializar en un tamaño correcto la lista de ficheros dentro de readFromDataInputStream (guardada como FileInfo[] fileList) que posteriormente se envía al cliente.

N: es el número indicado por nameLength.

M: es el número indicado por hashLength.

Mensaje: **peerdl <peer_nickname> <hash_substring> (opcode = 0x02) (TLV)**

Sentido de la comunicación: Cliente → Servidor de ficheros

Descripción: El Cliente pide a su Servidor bajo el nickname 'peer_nickname' descargar el fichero que empiece/tenga la subcadena hash que envía como parámetro.

Ejemplo:

Opcode (1 byte)	Longitud del hash (2 bytes)	Hash del fichero (n bytes)
0x02	0x00 0x0B	af3h1abcdef

Mensaje: **peerdIOk (opcode = 0x12) (TLV)**

Sentido de la comunicación: Servidor de ficheros → Cliente

Descripción: El Servidor le envía al cliente el archivo que acaba de solicitar.

Ejemplo:

Opcode (1 byte)	nameLength (4 bytes)	fileName (N bytes)	hashLength (4 bytes)	hashSubstring (M)	fileSize (8 bytes)
0x12	0x00 0x10 0x67 0x69	receta_queso. pdf	0x00 0x00 0x00 0x04	a1b2	...0x14 0x00 0x00

Mensaje: **peerdIFailed (opcode = 0xEE) (Control)**

Sentido de la comunicación: Servidor de ficheros → Cliente

Descripción: El Servidor le comunica al Cliente mediante un error que no se ha podido descargar el archivo deseado ya sea por un hash ambiguo (hay dos ficheros diferentes con ese hash) lo cual el cliente debería mandar un hash más largo o que no se ha encontrado ningún fichero con ese hash.

Ejemplo:

Opcode (1 byte)	TypeError (1 byte)
0xEE	0x01

Type Error varía entre FileNotFound (0x01) y AmbiguousHash (0x02), y el opcional ProgrammingError (0x03).

Mensaje: **peerdlGetChunk (opcode = 0x13) (TLV)**

Sentido de la comunicación: Cliente → Servidor de ficheros

Descripción: Tras haber recibido los metadatos del archivo en el mensaje peerdlOk, el Cliente comienza a solicitar fragmentos específicos del mismo para descargarlo por partes.

Ejemplo:

Opcode (1 byte)	hashLength (4 bytes)	hashSubstring (4 bytes)	fileOffset (8 bytes)
0x13	0x00 0x00 0x00 0x04	a1b2	0x00 0x00 0x00...

Mensaje: **peerdlSendChunk (opcode = 0x14) (TLV)**

Sentido de la comunicación: Servidor de ficheros → Cliente

Descripción: El Servidor responde a la solicitud enviando el desplazamiento y los datos binarios correspondientes.

Opcode (1 byte)	fileOffset (8 bytes)	dataLength (4 bytes)	data (N bytes)
0x14	0x00 0x00 0x00...	0x00 0x00 0x40 0x00	0x4A 0x56 0x31...

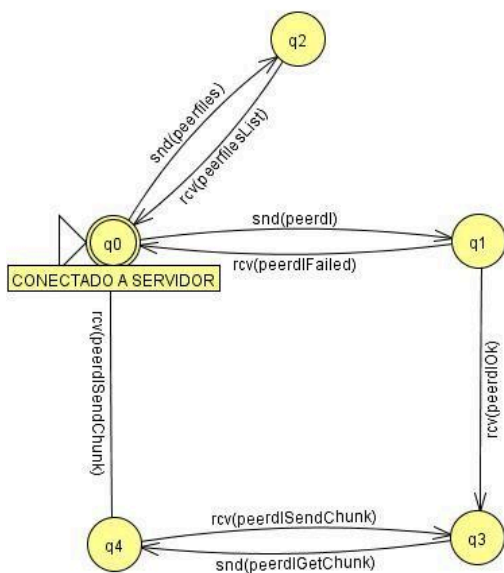
4. Autómatas de protocolo

Con respecto a los autómatas, hemos considerado las siguientes restricciones:

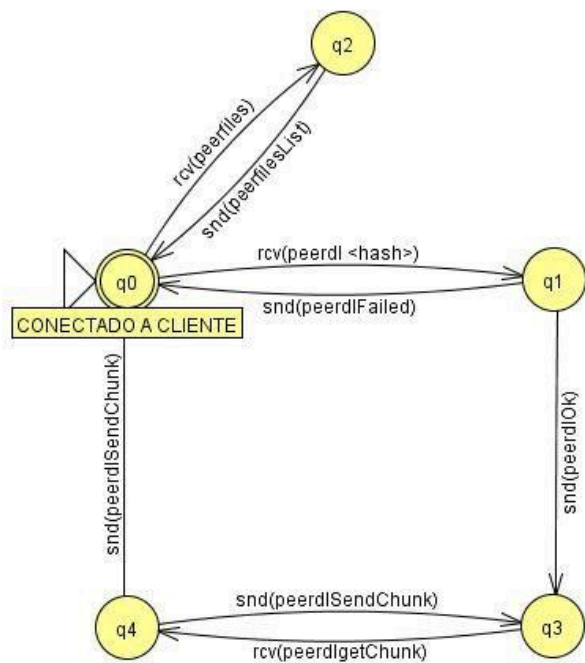
- Un usuario que desee conectarse al Directorio debe siempre pasar por el proceso de 'ping'.
- Un usuario no podrá enviar ningún mensaje, TCP o UDP, a no ser que haya sido aceptado por el Directorio previamente tras un 'pingOk'.
- Para poder comunicarte de forma P2P, al menos un Servidor debió de haberse conectado con 'ping' y haber hecho 'serve' y nunca 'quit'. El Cliente ha de haber sido aceptado por el Directorio con 'pingOk'.
- La comunicación TCP P2P no tiene métodos para romper la conexión: un subhilo causado por la conexión (NFServerThread) se iniciará el momento que un Cliente pida una operación, lo que dará lugar a los autómatas P2P que se verán justo

debajo.

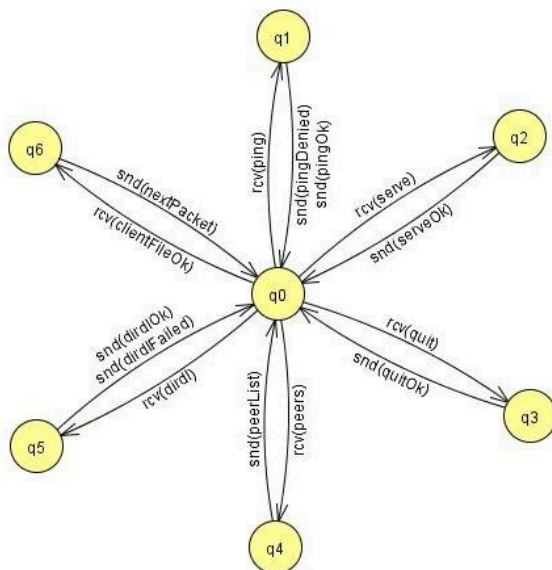
Autómata rol cliente de ficheros (NFConnector)



Autómata rol servidor de ficheros (NFServerThread)

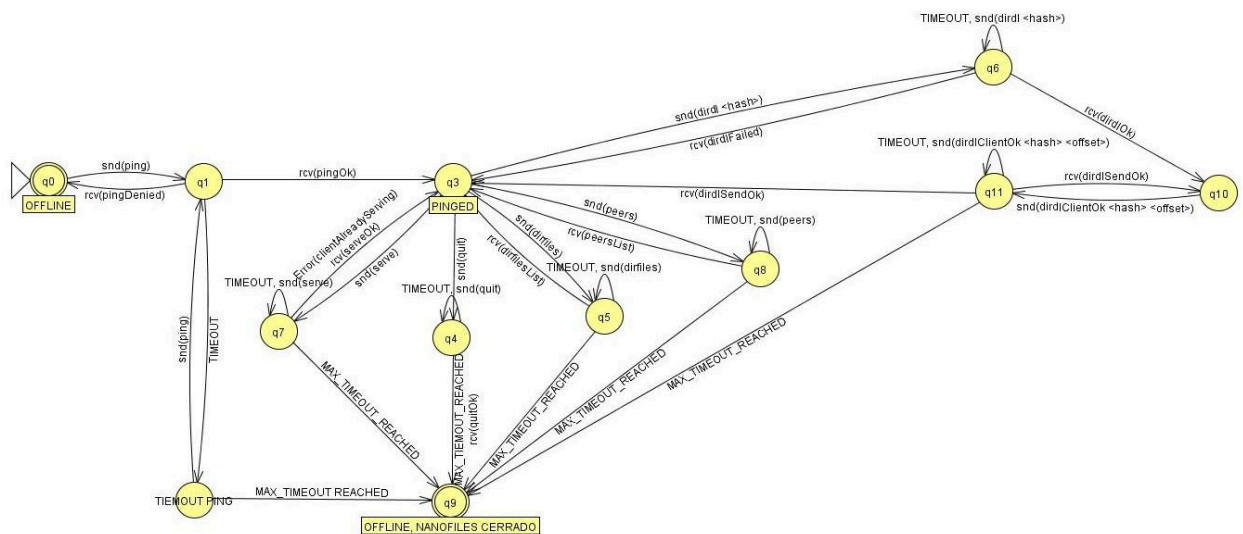


Autómata rol servidor de directorio



Autómata rol cliente de directorio

(Este es un poco extenso. Al hacer zoom mejora la calidad <3)



En este autómata, para acortar el uso de estados extras, sólo tenemos un *TIMEOUT PING* para diferenciarlo de volver al q0 offline, pero en el resto de estados que esperan tras enviar un mensaje (*snd(x)*), poseen una flecha que se apuntan a sí mismos y dicen “*TIMEOUT, snd(x)*” para mostrar que vuelve a sí mismo tras recibir un *TIMEOUT* y realizar un *snd(x)*. Luego, tras hacer 5 *TIMEOUTs* (*MAX_TIMEOUT*), va al estado final Offline que cierra la aplicación.

5. Ejemplo de intercambio de mensajes

Aquí hay un pequeño ejemplo de cómo podría ir una comunicación usando NanoFiles con el Directorio, y posteriormente una conversación entre Peers, con uno sirviendo como Servidor y otro como Cliente que pide ficheros.

//CLIENT-SIDE:

CLIENTE: Realiza un ping al Directorio con un identificador de protocolo erróneo.

operation: ping\n

protocol: 0\n

\n

DIRECTORIO: Responde negando la comunicación al no coincidir el protocolo.

operation: pingDenied\n

\n

CLIENTE: Realiza un ping con el protocolo correcto de la práctica.

operation: ping\n

protocol: 69696969\n

\n

DIRECTORIO: Confirma que el protocolo es válido.

operation: pingOk\n

\n

CLIENTE: Solicita la lista de ficheros que el directorio posee globalmente.

operation: dirfiles\n

\n

DIRECTORIO: Devuelve la lista con el número de archivos y sus detalles.

operation: dirfilesList\n

number_files: 2\n

file: hola.txt\n

size: 12242\n

hash_substring: a1b2c3\n

file: adios.pdf\n

size: 1953135\n

hash_substring: a1b3b4\n

\n

CLIENTE: Solicita descargar un fichero del directorio mediante una subcadena hash.

operation: dirdl\n

hash_substring: a1b2\n

\n

DIRECTORIO: Confirma que tiene el fichero y envía los metadatos para iniciar la descarga fragmentada.

operation: dirdlOk\n

file: hola.txt\n

size: 12242\n

hash_substring: a1b2c3\n

\n

CLIENTE: Solicita el primer fragmento del archivo (offset 0).

operation: dirdlClientOk\n

hash_substring: a1b2c3\n

offset: 0\n

\n

DIRECTORIO: Envía el fragmento de datos correspondiente.

operation: dirdlSendOk\n

offset: 0\n

data: <datos_binarios_del_fichero>\n

//FILESERVER-SIDE:

SERVIDOR: Intenta contactar con el directorio usando un protocolo erróneo.

operation: ping\n

protocol: 0\n

DIRECTORIO responde: Al no reconocer el protocolo, deniega la conversación.

operation: pingDenied\n

SERVIDOR: Envía el ping con el identificador correcto.

operation: ping\n

protocol: 69696969\n

DIRECTORIO: Confirma que el protocolo es compatible.

operation: pingOk\n

SERVIDOR envía: Alicia solicita registrarse para ofrecer sus archivos en el puerto 10000. Observamos que el registro se hace directamente con la operación serve, ya que el comando login ha sido eliminado del protocolo por eficiencia.

operation: serve\n

nickname: alicia\n

port: 10000\n

DIRECTORIO: El servidor guarda los datos de Alicia (IP detectada y puerto) y confirma el registro exitoso.

operation: serveOk\n

SERVIDOR : Eustaquio realiza el mismo proceso tras el ping.

operation: serve\n

nickname: eustaquio_roquefort\n

port: 10000\n

DIRECTORIO: Confirma el registro.

operation: serveOk

IP: 199.69.69.69

port: 10000

SERVIDOR: Eustaquio decide dejar de compartir archivos y pide ser eliminado de la lista de servidores activos.

operation: quit

nickname: eustaquio_roquefort

port: 10000

DIRECTORIO responde: Confirma que ha sido eliminado de las estructuras de datos.

operation: quitOk

//PEER-TO-PEER:

//CLIENTE (alicia) abre un socket TCP contra la IP 199.69.69.69 en el puerto 10000 de SERVIDOR (eustaquio_roquefort)

CLIENTE: pide la lista de los ficheros que Eustaquio tiene disponibles.
Opcode (1 byte): 0x01

SERVIDOR: devuelve la lista de ficheros que posee (supongamos que tiene 2 archivos).
Opcode (1 byte): 0x11
NumFiles (2 bytes): 0x00 0x02

//FICHERO 1:
nameLength (1 byte): 0x10
fileName (16 bytes): receta_queso.pdf
fileSize (8 bytes): 0x00 0x00 0x00 0x00 0x00 0x14 0x00 0x00
hashLength (1 byte): 0x04
hashSubstring (4 bytes): a1b2

//FICHERO2:
nameLength (1 byte): 0x09
fileName (9 bytes): memes.jpg
fileSize (8 bytes): 0x00 0x00 0x00 0x00 0x00 0x50 0x00 0x00
hashLength (1 byte): 0x04
hashSubstring (4 bytes): a1c3

CLIENTE: pide descargar un archivo que empieza por una subcadena hash ambigua:
Opcode (1 byte): 0x02
hashLength (1 byte): 0x02
hashSubstring (2 bytes): a1

SERVIDOR (eustaquio_roquefort): comunica que la subcadena es ambigua y no sabe a cuál se refiere:
Opcode (1 byte): 0xEE
error (1 byte): 2

CLIENTE: corrige y pide descargar un archivo que no existe:
Opcode (1 byte): 0x02
hashLength (1 byte): 0x03

hashSubstring (3 bytes): fff

SERVIDOR (eustaquio_roquefort): comunica que no concuerda con ningún fichero:

Opcode (1 byte): 0xEE

error (1 byte): 1

CLIENTE: pide descargar el archivo correcto proporcionando el hash exacto:

Opcode (1 byte): 0x02

hashLength (1 byte): 0x04

hashSubstring (4 bytes): a1b2

SERVIDOR: envía la confirmación seguida de los datos del fichero.

Opcode (1 byte): 0x12

DataLength (4 bytes): 0x00 0x14 0x00 0x00

FileData (N bytes): <datos_binarios_de_receta_queso.pdf>

6. Mejoras Implementadas

Como hemos mencionado en Introducción, las mejoras que hemos decidido implementar son Quit Avanzado, DirDL Avanzado, y por extensión, DirDL Básico.

DirDL es capaz de enviar mensajes desde dir-shared hasta el nf-shared del proceso NanoFiles que lo haya pedido. Envía los ficheros del tamaño que sea en paquetes pequeños de 40000 Bytes de contenido por cada uno, y muestra una barra de progreso que indica cómo va.

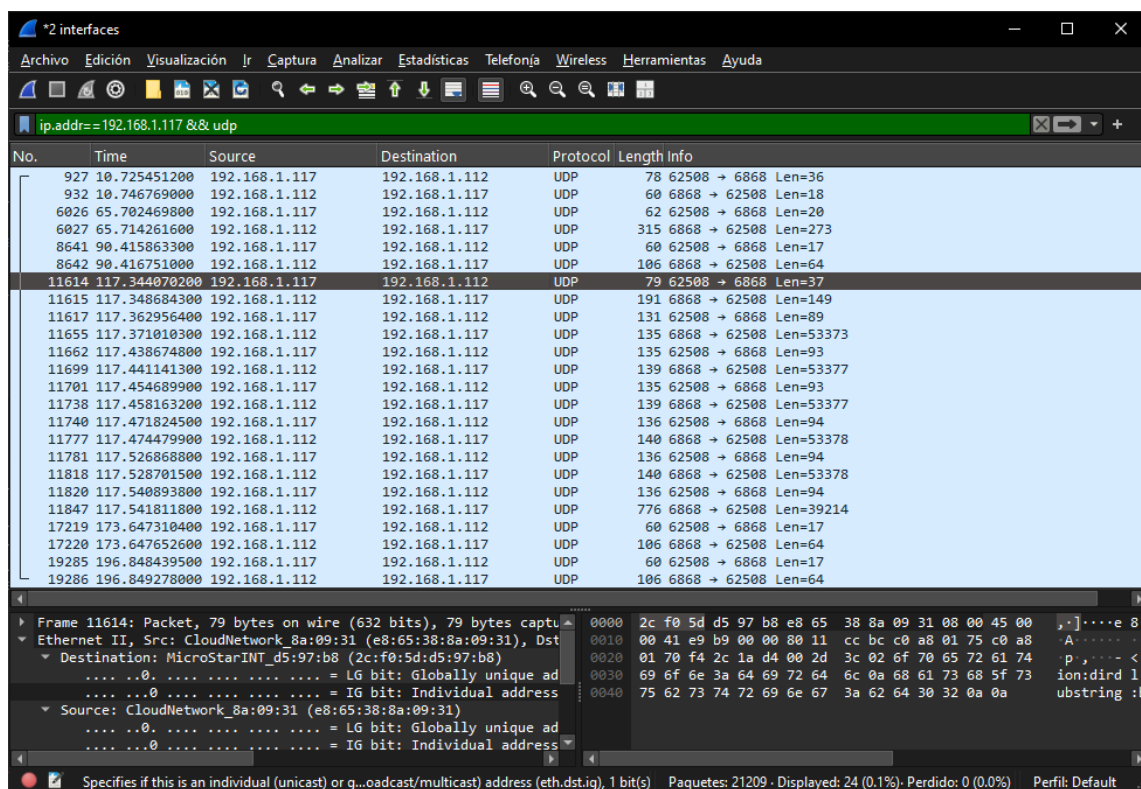
Para comprobarlo, puedes meter ficheros en dir-shared que puedan ser convertidos en base64, y pedir desde un NanoFiles la función dirdl <hash>, tras dirfiles.

Quit ahora tiene en cuenta si un NanoFiles ha sido conectado como un servidor de ficheros (ha hecho serve previamente), y envía un mensaje al Directorio para que tenga en cuenta que desea parar su ejecución, y por consiguiente, ser eliminado de los peers disponibles. Si ocurre TIMEOUTs demasiadas veces, simplemente cierra la aplicación sin

mandar el mensaje.

7.Captura de Pantalla de WireShark

En el ZIP entregado encontrarás la Captura WireShark (.pcapng) de la comunicación realizada. Aquí, para rellenar, te presentamos una captura de pantalla (literalmente una imagen) del resultado al ejecutar con Eclipse un Directory e interactuar con él a través de NanoFiles.



8. Conclusión

En este proyecto de la asignatura de Redes de Comunicaciones, hemos logrado implementar con éxito el sistema NanoFiles, cumpliendo con la funcionalidad básica exigida para establecer tanto un modelo cliente-servidor con el Directorio como una red P2P entre pares. A lo largo del desarrollo, hemos completado las operaciones obligatorias de registro, consulta y transferencia, integrando los comandos ping, dirfiles, peers, serve, peerfiles y peerdl, además de las mejores de dirdl y un mejor quit.

Durante la fase de diseño e implementación, nos enfrentamos a varios desafíos técnicos notables. Uno de los mayores retos fue el comprender cómo teníamos que modelar los mensajes ASCII y la estructura que queríamos que tuviera, ya que nos costó la mayor parte del tiempo.

Finalmente, esta práctica nos ha permitido consolidar los conceptos teóricos de la asignatura mediante su aplicación práctica. Hemos comprendido de forma tangible la diferencia arquitectónica entre diseñar un protocolo sobre UDP (donde nosotros mismos debíamos garantizar la confiabilidad, gestionar los temporizadores e implementar parada y espera) y trabajar sobre TCP (donde el transporte confiable ya está garantizado por la pila de protocolos subyacente).